

# Lecture 6: MapReduce Design Patterns II

DSL351 / DSP352

---

Amit Kumar Dhar

ED1, 406A

amitkdhar@iitbhlai

January 15, 2026

# Introduction

---

Today we move beyond basic summarization and filtering to handle more complex data relationships and performance issues.

- **Organization Patterns:** Reorganizing data layout (binning, partitioning).
- **Join Patterns:** Combining datasets (e.g., Users + Transactions).
- **Sorting Patterns:** Controlling the order of data processing.
- **Handling Skew:** Dealing with the "Curse of the Last Reducer".

# Organization Patterns

---

## 6. Binning Pattern

**Goal:** Organize records into different output files/folders based on a value.

- **Example:** Separate log data into 'ERROR', 'WARN', 'INFO' folders.
- **Standard MR:** Only one output directory.
- **Solution:** `MultipleOutputs` class.
- Can be done in Map-only (no shuffling) or Reducer.

# The MultipleOutputs Class

- Standard MapReduce produces files like `part-m-00000` in a single directory.
- `MultipleOutputs` allows a task to write to multiple files or even different directories.
- **Key Capabilities:**
  - **Named Outputs:** Pre-define output labels in the Job configuration.
  - **Dynamic Paths:** Determine the target file path at runtime based on record content.
  - **Flexibility:** Supports different key/value types for different outputs.
  - `mos.write(namedOutput, key, value, baseOutputPath)`
- **Arguments Explained:**
  - `namedOutput`: Logical name for the output (defined in Driver).
  - `key value`: The record to be written to the output.
  - `baseOutputPath`: The specific file or folder name (relative to the job's main output directory).
- **Usage:** Initialize in `setup()`, write using `mos.write()`, and close in `cleanup()`.

# Binning: Mapper with MultipleOutputs

```
1 public class BinningMapper extends Mapper<Object, Text, Text, NullWritable> {  
2     private MultipleOutputs<Text, NullWritable> mos;  
3  
4     protected void setup(Context context) {  
5         mos = new MultipleOutputs<>(context);  
6     }  
7  
8     public void map(Object key, Text value, Context context) ... {  
9         String log = value.toString();  
10        if (log.contains("ERROR")) {  
11            mos.write("bins", value, NullWritable.get(), "error-logs");  
12        } else if (log.contains("WARN")) {  
13            mos.write("bins", value, NullWritable.get(), "warn-logs");  
14        }  
15    }  
16  
17    protected void cleanup(Context context) {  
18        mos.close();  
19    }  
20 }
```

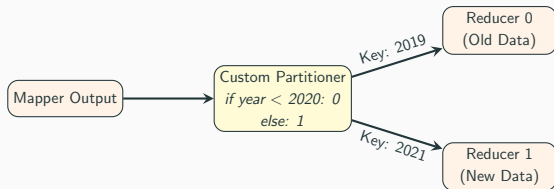
## 7. Partitioning Pattern

**Goal:** Control which Reducer processes which record.

- **Default:** 'HashPartitioner'  $\rightarrow hash(key) \% numReducers$ .
- **Problem:** What if we want all data for "Region A" in one file (Reducer 1) and "Region B" in another (Reducer 2)?
- **Solution:** Custom 'Partitioner'.



# Concept: Partitioning



# Partitioning: Custom Code

```
1 public class YearPartitioner extends Partitioner<IntWritable, Text> {
2
3     @Override
4     public int getPartition(IntWritable key, Text value, int numPartitions) {
5         // key is Year
6         int year = key.get();
7
8         if (numPartitions < 2) return 0; // Safety check
9
10        if (year < 2020) {
11            return 0; // Goes to Reducer 0
12        } else {
13            return 1; // Goes to Reducer 1
14        }
15    }
16 }
17
18 // In Driver:
19 job.setPartitionerClass(YearPartitioner.class);
20 job.setNumReduceTasks(2);
```

## 8. Total Order Sorting

**Goal:** Produce a globally sorted output file (or set of files).

- **Problem:** Hadoop sorts keys *within* a reducer, but not *across* reducers.
- **Solution:** Use a Partitioner that respects order.
- **Example:**
  - Reducer 0: Keys A-G
  - Reducer 1: Keys H-M
  - Reducer 2: Keys N-Z
- Requires understanding the distribution of keys (sampling) to create balanced partitions.

## Join Patterns

---

# The Challenge of Joins in MapReduce

**Goal:** Combine two large datasets based on a common key (Foreign Key).

- **SQL:** `SELECT * FROM A JOIN B ON A.id = B.id`
- **MapReduce:** We must bring matching records from A and B to the same place at the same time.
- **Strategies:**
  1. **Reduce-Side Join:** Safest, works for massive datasets.
  2. **Map-Side Join:** Faster, requires one dataset to fit in memory.

# Reduce-Side Join (Repartition Join)

## Mechanism:

- **Map:** Read multiple inputs. Tag each record with its source (e.g., "TableA", "TableB"). Emit (JoinKey, TaggedValue).
- **Shuffle:** Framework groups all values for JoinKey together.
- **Reduce:** Iterate through values, separate them into lists (List A, List B), and perform the cross-product.

**Pros:** Works for any size of data.

**Cons:** Heavy network traffic (Shuffle phase).

## Concept: Reduce-Side Join



## Reduce-Side Join: Mapper

```
1 public class JoinMapper extends Mapper<Object, Text, Text, Text> {
2     private Text outKey = new Text();
3     private Text outValue = new Text();
4
5     public void map(Object key, Text value, Context context) ... {
6         // Distinguish input based on file path or content
7         String filePath = ((FileSplit) context.getInputSplit()).getPath().getName();
8         String[] parts = value.toString().split(",");
9
10        if (filePath.contains("users")) {
11            outKey.set(parts[0]); // UserID
12            outValue.set("A:" + parts[1]); // Tag as User
13        } else {
14            outKey.set(parts[1]); // UserID in Transaction
15            outValue.set("B:" + parts[2]); // Tag as Txn Amount
16        }
17        context.write(outKey, outValue);
18    }
19 }
```



## Reduce-Side Join: Reducer

```
1 public class JoinReducer extends Reducer<Text, Text, Text, Text> {
2     public void reduce(Text key, Iterable<Text> values, Context context) ... {
3         ArrayList<String> listA = new ArrayList<>();
4         ArrayList<String> listB = new ArrayList<>();
5
6         for (Text val : values) {
7             String str = val.toString();
8             if (str.startsWith("A:")) listA.add(str.substring(2));
9             else if (str.startsWith("B:")) listB.add(str.substring(2));
10        }
11
12        // Inner Join Logic: Cross Product
13        for (String a : listA) {
14            for (String b : listB) {
15                context.write(key, new Text(a + "\t" + b));
16            }
17        }
18    }
19 }
```

# Optimization: Bloom Filter Join

**Scenario:** Reduce-Side Join where one table is large but has many non-matching keys.

- **Problem:** We shuffle all records, even those that won't match.
- **Solution:** Train a Bloom Filter on the keys of the smaller table.
- **Execution:**
  1. Compute Bloom Filter of Keys(SmallTable).
  2. Push to DistributedCache.
  3. Mappers of LargeTable check `filter.mightContain(key)`.
  4. If false, drop the record *before* shuffle.

## Map-Side Join (Replicated Join)

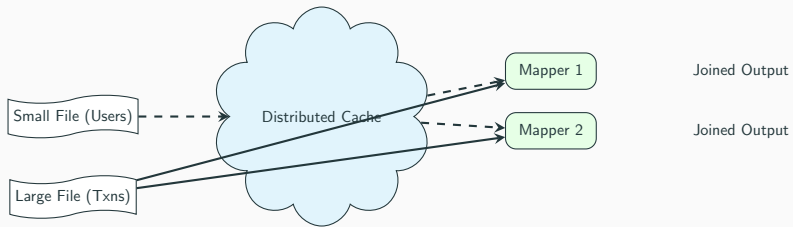
**Scenario:** One dataset (e.g., Users) is small enough to fit in memory; the other (Transactions) is huge. **Mechanism:**

- **DistributedCache:** Copy the small file to every Mapper node.
- **Setup:** Read the small file into a HashMap.
- **Map:** Stream the large file. Look up keys in the HashMap. Join locally.
- **No Reducer needed!**

**Pros:** Ultra-fast (no shuffle, no sort).

**Cons:** Limited by memory (Small dataset must fit in RAM).

## Concept: Map-Side Join



# Map-Side Join: Mapper

```
1 public class MapSideJoinMapper extends Mapper<Object, Text, Text, Text> {
2     private HashMap<String, String> usersMap = new HashMap<>();
3
4     protected void setup(Context context) {
5         URI[] files = context.getCacheFiles(); // Get cached file
6         // Read file and populate usersMap
7         BufferedReader br = new BufferedReader(new FileReader(files[0].getPath()));
8         String line;
9         while ((line = br.readLine()) != null) {
10             String[] parts = line.split(",");
11             usersMap.put(parts[0], parts[1]); // ID -> Name
12         }
13         br.close();
14     }
15
16     public void map(Object key, Text value, Context context) ... {
17         String[] parts = value.toString().split(","); // Transaction
18         String userId = parts[1];
19         String userName = usersMap.get(userId); // Local Lookup
20
21         if (userName != null) {
22             context.write(new Text(userName), value);
23         }
24     }
```

# Sorting Patterns

---

# Sorting in MapReduce

- **Built-in:** MapReduce sorts keys *before* passing them to the Reducer.
- **Limitations:**
  - Sorting is local to each Reducer.
  - Values associated with a key are *not* sorted.

# Total Order Sort: The Challenge

**Goal:** Produce globally sorted output across multiple output files.

- **Default Behavior:**

- HashPartitioner sends keys to reducers randomly.
- File 0: [Apple, Zebra], File 1: [Banana, Yak].
- Merging these files requires another sort.

- **Requirement:**

- Partition 0: keys  $< H$
- Partition 1:  $H \leq \text{keys} < N$
- Partition 2: keys  $\geq N$



# Total Order Sort: Implementation

**Solution:** TotalOrderPartitioner.

- Uses a binary tree (stored in a partition file) to route keys.
- **Step 1: Sampling** (Run before the main job).
- **InputSampler** reads a fraction of input to determine fair boundaries.

```
1 // Driver Code
2 InputSampler.Sampler<Text, Text> sampler =
3     new InputSampler.RandomSampler<>(0.1, 10000);
4
5 InputSampler.writePartitionFile(job, sampler);
6
7 job.setPartitionerClass(TotalOrderPartitioner.class);
```

## Secondary Sort

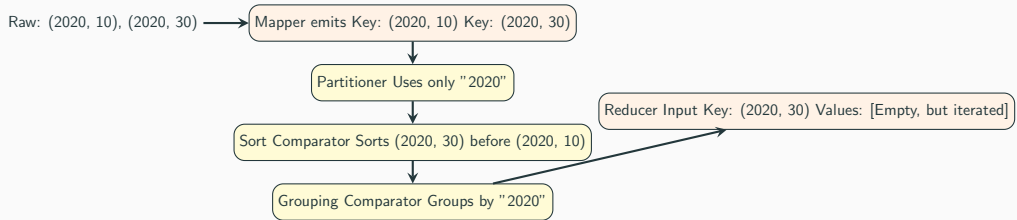
**Problem:** We want to sort values *for a given key*.

- Input: Year, Temperature.
- Goal: For each Year, process Temperatures in descending order.
- Standard MR receives: 2020 -> [10, 5, 20, 15] (Arbitrary order).

### **Solution: Value-to-Key Conversion**

1. **Composite Key:** Move the value into the key. New Key: (Year, Temp).
2. **Custom Partitioner:** Partition only by Natural Key (Year) so all records for 2020 go to same reducer.
3. **Key Comparator:** Sort by Composite Key (Year ASC, Temp DESC).
4. **Grouping Comparator:** Group by Natural Key (Year) so Reducer sees one group.

## Concept: Secondary Sort



## Secondary Sort: Composite Key

We create a Custom Writable that holds both the *Natural Key* (Year) and the *Secondary Key* (Temperature).

```
1 public class DateTemperaturePair
2     implements WritableComparable<DateTemperaturePair> {
3
4     private Text yearMonth;
5     private IntWritable temperature;
6
7     public int compareTo(DateTemperaturePair pair) {
8         int cmp = this.yearMonth.compareTo(pair.getYearMonth());
9         if (cmp == 0) {
10             // Sort Temperature DESC
11             return -1 * this.temperature.compareTo(pair.getTemperature());
12         }
13         return cmp;
14     }
15 }
```

## Secondary Sort: Partitioner

We must ensure that all records for the same *Natural Key* go to the same reducer, ignoring the secondary key.

```
1 public class NaturalKeyPartitioner
2     extends Partitioner<DateTemperaturePair, Text> {
3
4     @Override
5     public int getPartition(DateTemperaturePair pair,
6                             Text value, int numPartitions) {
7         // Hash ONLY the Year, not the Temperature
8         return Math.abs(pair.getYearMonth().hashCode() % numPartitions);
9     }
10 }
```

# Handling Skewed Data

---

# The Problem of Skew

**Scenario:** "The Curse of the Last Reducer"

- You have 100 reducers.
- 99 finish in 5 minutes.
- 1 runs for 5 hours.

**Cause:** Uneven distribution of keys.

- Example: Word Count on Wikipedia. The word "the" appears millions of times.
- One reducer gets all "the" keys → Overloaded.

# Skew Mitigation Strategy: Salting

**Technique:** Add a random prefix/suffix to the key to split the heavy key.

## 1. Stage 1 (Salting):

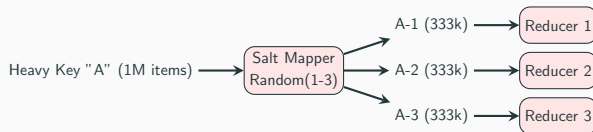
- Mapper: If Key is "the", change it to "the-1", "the-2" ... "the-10".
- Reducer: Counts "the-1", "the-2" separately. (Load is spread across reducers).

## 2. Stage 2 (Aggregation):

- Mapper: Read output of Stage 1. Remove salt ("the-1" → "the").
- Reducer: Sum up the partial counts.



## Concept: Salting for Skew



Parallel Processing!  
No single bottleneck.

# Skewed Join Optimization

**Scenario:** Joining Table A (Uniform) and Table B (Skewed on Key 'K').

- **Method:**

1. Isolate the skewed keys (e.g., 'K') from Table B.
  2. **Replicate** the skewed keys of Table A to all reducers (or map-side join for just these keys).
  3. **Randomly distribute** the skewed records of Table B (Salting).
  4. Perform standard join on the non-skewed data.
- Modern frameworks (Spark/Hive) often do "Skew Join" automatically by detecting large partitions.

# Summary

---

| Problem               | Pattern/Solution                 |
|-----------------------|----------------------------------|
| Binning               | MultipleOutputs in Mapper        |
| Partitioning          | Custom Partitioner logic         |
| Joining Large + Large | Reduce-Side Join (Repartition)   |
| Joining Large + Small | Map-Side Join (Replicated/Cache) |
| Global Sorting        | TotalOrderPartitioner + Sampling |
| Value Sorting         | Secondary Sort (Composite Key)   |
| Data Skew             | Salting (Two-stage aggregation)  |

**Questions?**